DBMS TAE-2 MINI-PROJECT

COLLEGE EVENT REGISTRATION SYSTEM

A Database-Connected Application Using Python, Streamlit and SQLite

DETAIL	INFORMATION
College / Institute	G. H. Raisoni International Skill Tech University
Department	B.Tech (AI-DS)
Subject	Database Management Systems (DBMS)
Activity	TAE-2 Mini-Project
Academic Year	2026–2027
Faculty Guide	Sandip Kahate

Team Members

Sr. No.	Name	Roll Number
1	VAIDEHI SASWADE	TB4-33
2	SHRUTIKA BANDI	TB4-29
3	SRUSHTI MAPARI	TB4-32

Submitted in partial fulfilment of the requirements for the DBMS TAE-2 activity.

CERTIFICATE

This is to certify that the mini-project report entitled **“College Event Registration System”** is submitted by the following students of B.Tech (AI-DS) at G. H. Raisoni International Skill Tech University, Pune, as part of the Database Management Systems (DBMS) TAE-2 activity for the academic year 2026–2027.

Sr. No.	Name	Roll Number
1	Vaidehi Saswade	TB4-33
2	Shrutika Bandi	TB4-29
3	Srushti Mapari	TB4-32

The project demonstrates relational database design, database connectivity, and the basic Create, Read, Update and Delete (CRUD) operations using Python, Streamlit and SQLite.

_____	_____
Faculty Guide	Department / Project Approval
Sandip Kahate	G. H. Raisoni International Skill Tech University

Date: _____ Place: Pune

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our faculty guide, Sandip Kahate, for his guidance and support during the preparation of our DBMS TAE-2 mini-project, “College Event Registration System.” His explanations of database concepts helped us understand how a database-connected application can be designed in a structured way.

We are thankful to G. H. Raisoni International Skill Tech University, Pune, and the School of Engineering and Technology for providing an academic environment in which we could learn and apply database management concepts. We also appreciate the support of our department and classmates during project planning and discussion.

This project gave us an opportunity to study relational tables, primary keys, foreign keys, SQL queries, database connectivity and CRUD operations. We also learned the importance of validating user input and keeping related records consistent.

Finally, we thank our families for their encouragement and support throughout our academic work.

Team Members

1. Vaidehi Saswade — TB4-33
2. Shrutika Bandi — TB4-29
3. Srushti Mapari — TB4-32

INDEX

Chapter	Title	Page
—	Cover Page	1
—	Certificate	2
—	Acknowledgement	3
—	Index	4
01	Abstract and Project Overview	5
02	Introduction and Background	6
03	Problem Definition and Objectives	7
04	Scope, Users and Feasibility	8
05	Project Architecture	9
06	System Workflow and Use Cases	10
07	Entity Relationship Diagram	11
08	Database Design and Data Dictionary	12–13
09	Normalization	14
10	Software and Hardware Requirements	15
11	Modules and Functional Description	16
12	CRUD Operations	17
13	SQL Queries and Database Connectivity	18
14	Input Validation and Data Integrity	19
15	Testing Plan and Expected Results	20
16	Sample Data and Demonstration Scenarios	21
17	User Interface / Screenshot Checklist	22
18	Installation and Execution Guide	23
19	Security, Limitations and Future Scope	24
20	Conclusion and Learning Outcomes	25
—	References	26
—	Appendix: Viva Questions	27

Note: Insert genuine screenshots from the running application in the screenshot spaces/checklist. Do not describe a feature as tested until it has been demonstrated in the actual application.

CHAPTER 01

ABSTRACT AND PROJECT OVERVIEW

The College Event Registration System is a small database-driven application intended to maintain college event details, student information and registrations. Colleges conduct technical, cultural, sports and social activities throughout the academic year. When records are kept only in notebooks or separate spreadsheets, it may become difficult to track registrations, update event information and check which students have registered for an event.

This project proposes a browser-based interface built with Python and Streamlit, connected to an SQLite relational database. The system is designed around three main entities: students, events and registrations. The registrations table links students and events and represents the many-to-many relationship between them.

The application is intended to demonstrate the four basic database operations: Create, Read, Update and Delete. A user can add an event or student, view saved records, edit information and remove test records. Registration records connect a student with an event and can include a registration date and status.

The report explains the problem, objectives, scope, architecture, database schema, relationships, normalization, SQL operations, validation, testing approach and future enhancements. It is an academic mini-project and should be demonstrated with the actual application and genuine screenshots before final submission.

Project at a glance

Item	Details
Project name	College Event Registration System
Course / Activity	DBMS TAE-2 Mini-Project
Database	SQLite relational database
Application language	Python
User interface	Streamlit browser-based interface
Main entities	Students, Events, Registrations
Core functions	Create, Read, Update and Delete records
Purpose	Organised management of event and student registration information

Keywords: DBMS, SQLite, Python, Streamlit, relational database, primary key, foreign key, CRUD, SQL joins.

CHAPTER 02

INTRODUCTION AND BACKGROUND

2.1 Background

A college may arrange seminars, workshops, coding competitions, cultural programmes, sports activities and guest lectures. Every event has information such as its title, category, date, venue and available capacity. Students may register for one event or several events, and one event may receive registrations from many students.

A manual registration process can work for a small activity, but repeated data entry becomes difficult as the number of students and events increases. Staff may need to search multiple files to answer simple questions, such as which students registered for a workshop or how many registrations an event has received.

2.2 Need for the system

A central database gives the college a consistent place to store information. Student details are entered once and can be linked to different events through registration records. Event information can be updated without rewriting every registration. A database also makes it easier to view, filter and count records.

2.3 Introduction to DBMS

A Database Management System (DBMS) is software used to define, store, retrieve and manage data. A relational DBMS organises information into tables made of rows and columns. SQL is used to create tables and perform operations on the stored records. Primary keys identify records uniquely, while foreign keys connect related tables.

2.4 Why Python, Streamlit and SQLite?

Technology	Reason for selection
Python	Readable programming language for application logic and database access.
Streamlit	Provides a simple web interface using Python, suitable for an academic demonstration.
SQLite	Stores data in a local database file and does not require a separate database server.
sqlite3	Python standard-library module for connecting to SQLite.
Pandas (optional)	Can be used to display query results in a convenient table format.

2.5 Intended outcome

The intended outcome is a small working application through which a user can manage student and event records and maintain registrations in a relational database. The exact features available in the final implementation should be confirmed by running the application.

CHAPTER 03

PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem statement

A college requires a simple and organised method to store event information, student details and registrations. The system should connect a Python-based user interface to a relational database and support the basic CRUD operations while maintaining valid relationships between students and events.

3.2 Problems in a manual process

• Duplicate student details may be entered in different event lists.
• Event dates or venues may be changed in one document but not another.
• Counting registrations manually can take time and may produce mistakes.
• Searching for one student's registrations may require checking multiple sheets.
• Records can be misplaced or become inconsistent when several copies exist.
• It may be difficult to preserve a clear relationship between a student and a specific event.

3.3 Objectives

1. Design a relational database for students, events and registrations.
2. Connect the application to SQLite using Python.
3. Implement Create, Read, Update and Delete operations for the main records.
4. Use primary keys and foreign keys to maintain relationships.
5. Provide a simple browser-based interface using Streamlit.
6. Validate important input fields before saving records.
7. Demonstrate SQL queries, joins and basic database concepts.
8. Document the design, workflow, testing approach and possible improvements.

3.4 Expected benefits

Benefit	Explanation
Centralised records	Students, events and registrations are stored in related tables.
Reduced duplication	Student information can be stored once and referenced by ID.
Faster retrieval	SQL SELECT queries can display records quickly.
Easier updates	An event or student record can be edited in one place.
Clear relationships	Foreign keys link registrations to valid students and events.
Academic learning	The project applies DBMS concepts in a practical example.

3.5 Success criteria

The project can be considered demonstrable when the application connects to the database, displays existing records, saves a new test record, updates that record, deletes a safe test record and displays registrations linked to valid student and event IDs. These checks should be performed on the actual running application.

CHAPTER 04

SCOPE, USERS AND FEASIBILITY

4.1 Scope of the project

The project focuses on event and student record management in a college environment. Its intended scope includes storing event details, maintaining student details, creating student-to-event registrations, viewing saved information and editing or deleting records through a simple interface.

In-scope functions

• Add and view event records.
• Add and view student records.
• Update details for an existing event or student.
• Delete suitable test records.
• Create and view registrations that connect students and events.
• Store data in an SQLite database file.
• Show database records through Streamlit tables or forms.

4.2 Out of scope / limitations

The basic academic version does not necessarily include secure login, role-based permissions, online payment, QR-code entry, automatic email or SMS notifications, cloud multi-user deployment, or integration with a college ERP. These features can be considered for future development if required by the project guide.

4.3 Intended users

User	Possible use
Event coordinator	Create event records and update event information.
Student / registration operator	Enter student details and create event registrations.
Department staff	View event lists and registration information.
Project evaluator	Verify the CRUD operations and database relationships during demonstration.

4.4 Feasibility study

Feasibility	Assessment
Technical	Python, Streamlit and SQLite are suitable for a small local demonstration and are widely used for learning.
Economic	The software stack can be used without purchasing a database server for the basic local project.
Operational	Forms and tables provide a simple interaction model that can be demonstrated by students and staff.
Schedule	The project can be built in stages: schema, database connection, CRUD screens, registration flow and testing.

4.5 Assumptions

The system assumes that each student has a unique student ID, each event has a unique event ID, and each registration refers to an existing student and event. Input data is assumed to be entered by an authorised operator in the demonstration environment. Real personal information should not be used for sample testing.

CHAPTER 05

PROJECT ARCHITECTURE

5.1 Overview

The application uses a simple layered structure. A user interacts with Streamlit in a browser. Python code receives the submitted form values, validates them, sends SQL statements through the sqlite3 module and displays the results returned by SQLite.

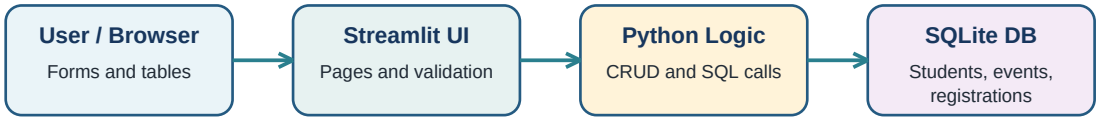


Figure: High-level application architecture

5.2 Components and responsibilities

Component	Responsibility
Presentation layer	Streamlit pages, forms, buttons, tables and status messages.
Application logic	Checks input values and decides which database operation should run.
Database connectivity	Opens an SQLite connection and executes SQL statements.
Database schema	Defines students, events and registrations tables with keys and constraints.
Query result display	Shows saved records and messages after an operation.

5.3 Data flow

1. The user opens the application in a browser.
2. The user selects the Events, Students or Registrations section.
3. The application displays a form or a list of existing records.
4. The user enters information and submits the form.
5. Python validates the values and executes the appropriate SQL statement.
6. SQLite saves or retrieves the data.
7. The interface refreshes and displays the result.

5.4 Design decisions

SQLite is suitable for a small project because the database is stored in a file and is easy to distribute with the application. Streamlit allows a user interface to be created in Python without building a separate frontend. The application should use parameterised SQL queries rather than joining raw user input into SQL text.

CHAPTER 06

SYSTEM WORKFLOW AND USE CASES

6.1 Application workflow

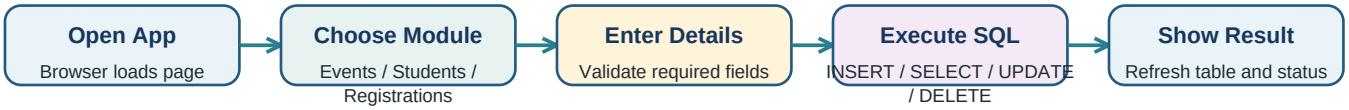


Figure: Typical user-to-database workflow

6.2 Main use cases

Use case	Actor	Expected action
Add event	Coordinator	Enter event name, category, date, venue and capacity; save the record.
View events	Any permitted user	Display event records stored in the database.
Update event	Coordinator	Select an event and edit its allowed details.
Delete event	Coordinator	Remove a test event when no dependent registration prevents deletion.
Add student	Registration operator	Enter student name, email, department and year of study.
Register for event	Registration operator	Select a valid student and event and create a registration.
View registrations	Staff	Display registration records, optionally joined with student and event names.

6.3 Typical registration sequence

A registration begins by selecting an existing student and an existing event. The application should check that both selections are valid, then insert the IDs into the registrations table along with the registration date and status. The interface can then show a confirmation and refresh the registration list.

6.4 Error handling

If required information is missing, the interface should show a clear validation message. If a student or event ID does not exist, the database relationship should prevent an invalid registration. If a record cannot be deleted because other records depend on it, the application should show a friendly message instead of displaying a raw database error.

6.5 User experience considerations

Forms should use clear field labels, suitable defaults and simple confirmation messages. Tables should show the most useful columns first. Destructive actions should be demonstrated with test data. For a classroom demo, the interface should make it easy to show each CRUD operation without navigating through too many pages.

CHAPTER 07

ENTITY RELATIONSHIP DIAGRAM

7.1 Main entities

The design uses three principal entities. STUDENTS stores student details, EVENTS stores event details, and REGISTRATIONS records the relationship between a student and an event. This structure avoids storing a list of event names inside each student row or a list of student names inside each event row.



Figure: Simplified ER relationship

7.2 Entity descriptions

Entity	Example attributes	Purpose
STUDENTS	student_id (PK), full_name, email, department, year_of_study	Stores each student's details.
EVENTS	event_id (PK), event_name, category, event_date, venue, capacity, description	Stores event information.
REGISTRATIONS	registration_id (PK), student_id (FK), event_id (FK), registration_date, status	Connects a student to an event and stores registration information.

7.3 Relationships and cardinality

One student can have multiple registration records, so STUDENTS to REGISTRATIONS is a one-to-many relationship. One event can also have multiple registration records, so EVENTS to REGISTRATIONS is another one-to-many relationship. Together, these relationships represent a many-to-many relationship between students and events.

7.4 Primary and foreign keys

A primary key uniquely identifies each row. For example, student_id identifies one student and event_id identifies one event. A foreign key is a field that references a key in another table. In REGISTRATIONS, student_id references STUDENTS and event_id references EVENTS. These relationships help prevent registrations from pointing to records that do not exist.

7.5 Design notes

The exact constraints depend on the implemented schema. A UNIQUE constraint on (student_id, event_id) may be used if the same student should not register for the same event more than once. Foreign-key enforcement should be enabled for SQLite connections. Deletion rules should be chosen carefully to avoid accidentally removing valid registration history.

CHAPTER 08

DATABASE DESIGN AND DATA DICTIONARY

8.1 Relational schema

The following schema is a recommended structure for the project and follows the entities described in the original report. Column types and constraints should be checked against the actual SQL file used in the application before submission.

```
STUDENTS(
  student_id       INTEGER PRIMARY KEY,
  full_name        TEXT NOT NULL,
  email            TEXT,
  department       TEXT,
  year_of_study    INTEGER
)

EVENTS(
  event_id         INTEGER PRIMARY KEY,
  event_name       TEXT NOT NULL,
  category         TEXT,
  event_date       TEXT NOT NULL,
  venue            TEXT,
  capacity         INTEGER,
  description      TEXT
)

REGISTRATIONS(
  registration_id  INTEGER PRIMARY KEY,
  student_id       INTEGER NOT NULL REFERENCES students(student_id),
  event_id         INTEGER NOT NULL REFERENCES events(event_id),
  registration_date TEXT,
  status           TEXT
)
```

8.2 Students table

Column	Suggested type	Purpose / rule
student_id	INTEGER	Primary key; unique student identifier.
full_name	TEXT	Student's name; required.
email	TEXT	Contact email, if collected.
department	TEXT	Academic department or branch.
year_of_study	INTEGER	Current year of study, if used.

8.3 Events table

Column	Suggested type	Purpose / rule
event_id	INTEGER	Primary key; unique event identifier.
event_name	TEXT	Event title; required.
category	TEXT	Technical, cultural, sports or other category.
event_date	TEXT	Event date, preferably stored in ISO format YYYY-MM-DD.
venue	TEXT	Location where the event will be held.
capacity	INTEGER	Optional maximum number of registrations.

Column	Suggested type	Purpose / rule
description	TEXT	Optional event summary.

CHAPTER 08 (CONTINUED)

DATA DICTIONARY AND RELATIONSHIP RULES

8.4 Registrations table

Column	Suggested type	Purpose / rule
registration_id	INTEGER	Primary key; identifies one registration.
student_id	INTEGER	Foreign key to STUDENTS.student_id.
event_id	INTEGER	Foreign key to EVENTS.event_id.
registration_date	TEXT	Date the registration was created.
status	TEXT	Example values: Registered, Cancelled, Attended; use the values supported by the app.

8.5 Example relationship query

The following SQL query joins registrations with students and events so the output can show names instead of only numeric IDs:

```
SELECT s.full_name, e.event_name,
       r.registration_date, r.status
FROM registrations AS r
JOIN students AS s
  ON r.student_id = s.student_id
JOIN events AS e
  ON r.event_id = e.event_id
ORDER BY r.registration_date DESC;
```

8.6 Integrity rules

- Each table should have a primary key.
- Required fields should use NOT NULL where appropriate.
- Foreign keys should link registrations to valid students and events.
- Capacity should not be negative if the field is used.
- Dates should be entered consistently and validated by the application.
- Duplicate registrations can be prevented with a UNIQUE constraint on student_id and event_id, if that rule is required.

8.7 Sample records (illustrative only)

student_id	full_name	department	year_of_study
1	Aarav Patil	AI-DS	3
2	Neha Sharma	Computer Engineering	2

event_id	event_name	category	venue
1	Python Workshop	Technical	Lab 2
2	Annual Cultural Day	Cultural	Auditorium

Use fictional records only for the report and demonstration. Replace these examples with screenshots or output from the actual application when required by the faculty guide.

CHAPTER 09

DATABASE NORMALIZATION

Normalization is a database design process used to reduce unnecessary repetition and prevent insertion, update and deletion anomalies. The three main normal forms discussed here are First Normal Form (1NF), Second Normal Form (2NF) and Third Normal Form (3NF).

9.1 First Normal Form (1NF)

Rule: each cell should contain a single value and the table should not store repeating groups in one column. For example, a student row should not contain a comma-separated list of event names. Each student-event link should be stored as a separate row in REGISTRATIONS.

Poor design	Normalised approach
student_id | name | events = 'Workshop, Sports Day'	Store the student once in STUDENTS and create two rows in REGISTRATIONS.

9.2 Second Normal Form (2NF)

Rule: a table must be in 1NF and each non-key attribute must depend on the whole candidate key, not only part of a composite key. The suggested design uses single-column primary keys for students, events and registrations, so partial dependency is avoided in those tables. If a composite key is used for registrations, attributes must depend on the complete key.

9.3 Third Normal Form (3NF)

Rule: a table must be in 2NF and non-key attributes should not depend transitively on the primary key. For example, event_name and venue belong in EVENTS, not repeated in every registration row. A registration should store event_id and student_id, then retrieve the descriptive details through joins.

9.4 Benefits of normalisation

- Reduces repeated data.
- Makes updates easier because details are stored in one place.
- Helps prevent inconsistent event names or student details.
- Supports clear relationships using keys.
- Makes joins and reports more reliable.

9.5 Practical consideration

Normalisation should match the actual application needs. A small project does not need an unnecessarily large number of tables. The three-table design is enough to demonstrate the many-to-many relationship between students and events. Additional tables may be introduced later for organisers, departments, venues or attendance if the scope expands.

CHAPTER 10

SOFTWARE AND HARDWARE REQUIREMENTS

10.1 Software requirements

Category	Requirement	Purpose
Operating system	Windows 10/11, Linux or macOS	Runs Python and the application.
Programming language	Python 3.x	Application logic and database operations.
UI framework	Streamlit	Creates browser-based pages and forms.
Database	SQLite 3	Stores relational records in a local file.
Connectivity module	sqlite3	Connects Python to SQLite.
Editor	VS Code, PyCharm or another editor	Creates and edits project files.
Browser	Chrome, Edge or Firefox	Displays the Streamlit application.
Optional library	Pandas	Displays query results in tabular form.

10.2 Hardware requirements

Component	Minimum for a basic demo	Recommended
Processor	Dual-core CPU	Modern dual-core or better
RAM	4 GB	8 GB or more
Storage	About 500 MB free	1–2 GB free for Python and project tools
Display	1280 × 720	1920 × 1080
Internet	Needed for initial package installation if not already installed	Stable connection for setup and documentation

10.3 Project files

A simple project folder may contain app.py, database.db (created when the application runs), requirements.txt and README.md. If the project is split into modules, it may also contain database.py, schema.sql and separate page or utility files. Use the files that actually exist in your project rather than adding names only for the report.

10.4 Dependencies

```
streamlit
# sqlite3 is included with Python
# pandas is optional if used by the application
```

10.5 Environment notes

Package versions can differ by computer. It is good practice to record the Python and Streamlit versions used for the final demonstration. If an application fails to start, check that Python is installed, the correct project folder is open, required packages are installed and the terminal is using the expected Python environment.

CHAPTER 11

MODULES AND FUNCTIONAL DESCRIPTION

11.1 Dashboard / home page

The home page can introduce the system and display summary information such as the total number of events, total students and total registrations. These values should be calculated from database queries rather than typed manually, so that they reflect saved records.

11.2 Event management module

This module manages event information. An event form can collect the event name, category, date, venue, capacity and description. The event list should show existing events and provide controls to edit or remove an appropriate record. Required values should be checked before the database is changed.

11.3 Student management module

This module stores student details such as full name, email, department and year of study. A table can display saved student records. The system should use a stable student_id to link the student with registrations. If email is used as a unique identifier, the rule should be defined clearly and enforced consistently.

11.4 Registration management module

The registration module connects one student to one event. The interface should allow the operator to choose from existing students and events. After submission, the registration is saved and shown in the registration list. The system can display student and event names by joining the related tables.

11.5 Search and filtering

If implemented, search and filters can help users locate an event by name or category, find a student by name, or filter registrations by event or status. Filtering should be performed using safe SQL parameters or by filtering the result set in a controlled way.

11.6 Validation messages

The interface should tell the user whether a record was saved, updated or deleted. If required values are missing, a field is invalid or a foreign-key relationship fails, the application should display a simple explanation. Avoid exposing internal database paths or lengthy error traces to end users.

11.7 Module summary

Module	Main database activity
Dashboard	SELECT and aggregate queries
Events	INSERT, SELECT, UPDATE, DELETE
Students	INSERT, SELECT, UPDATE, DELETE
Registrations	INSERT and SELECT; optionally update status or delete a test registration
Validation	Checks input and reports constraint errors

CHAPTER 12

CRUD OPERATIONS

CRUD stands for Create, Read, Update and Delete. These four operations form the basic record-management functions of the College Event Registration System.

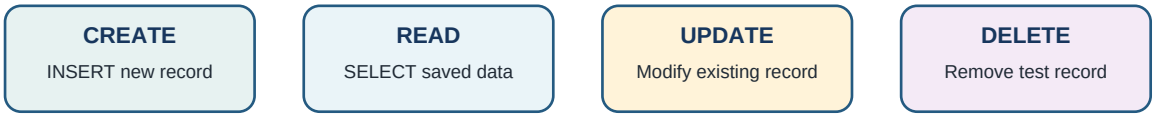


Figure: Four core database operations

12.1 Create — INSERT

The Create operation adds a new row to a table. For example, an event record can be inserted using the following parameterised SQL statement:

```
INSERT INTO events
(event_name, category, event_date, venue, capacity)
VALUES (?, ?, ?, ?, ?);
```

12.2 Read — SELECT

The Read operation retrieves existing records. A basic query displays all events in date order:

```
SELECT event_id, event_name, category, event_date, venue
FROM events
ORDER BY event_date;
```

12.3 Update — UPDATE

The Update operation changes the values of an existing record. The WHERE condition is important because it identifies which event should be changed:

```
UPDATE events
SET venue = ?
WHERE event_id = ?;
```

12.4 Delete — DELETE

The Delete operation removes a record. Demonstrate deletion only with a test event or registration. If a record is referenced by another table, a foreign-key rule may block the deletion until dependent records are handled safely.

```
DELETE FROM events
WHERE event_id = ?;
```

12.5 CRUD demonstration table

Operation	SQL	Demonstration
Create	INSERT	Add a test event and confirm it appears in the list.
Read	SELECT	Display the saved event and student tables.
Update	UPDATE	Change a test event's venue and verify the new value.
Delete	DELETE	Remove a safe test record and verify it no longer appears.

CHAPTER 13

SQL QUERIES AND DATABASE CONNECTIVITY

13.1 Connecting Python to SQLite

Python's sqlite3 module provides a connection to an SQLite database file. The application should use a consistent database path and close connections appropriately. The following is an illustrative connection example; adapt the path and function names to the actual application.

```
import sqlite3

DB_PATH = "database.db"

def get_connection():
    connection = sqlite3.connect(DB_PATH)
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA foreign_keys = ON")
    return connection
```

13.2 Parameterised queries

User input should be passed as parameters. This avoids treating text entered by a user as part of the SQL command and helps protect against SQL injection.

```
with get_connection() as conn:
    conn.execute(
        "INSERT INTO students (full_name, email) VALUES (?, ?)",
        (full_name, email)
    )
```

13.3 Join query

```
SELECT s.full_name, e.event_name, r.status
FROM registrations r
JOIN students s ON s.student_id = r.student_id
JOIN events e ON e.event_id = r.event_id;
```

13.4 Count registrations per event

```
SELECT e.event_name, COUNT(r.registration_id) AS total
FROM events e
LEFT JOIN registrations r ON r.event_id = e.event_id
GROUP BY e.event_id, e.event_name
ORDER BY total DESC;
```

13.5 Count students

```
SELECT COUNT(*) AS total_students
FROM students;
```

13.6 Transaction handling

A transaction groups database changes into one logical unit. SQLite connections used as context managers can commit a successful transaction and roll back if an error occurs. For a simple insert, update or delete, transaction handling helps ensure that incomplete changes are not accidentally left behind.

13.7 Query result display

The application can fetch rows and display them with Streamlit's table or dataframe components. The exact display method depends on the current implementation. The report should include genuine output screenshots to show that the queries run successfully.

CHAPTER 14

INPUT VALIDATION AND DATA INTEGRITY

14.1 Why validation matters

Validation checks that information is complete and in an acceptable format before it is saved. It improves the user experience and reduces invalid data. Database constraints provide a second layer of protection, because data can sometimes reach the database through a different code path.

14.2 Suggested validation rules

Field	Suggested validation
Student name	Required; trim extra spaces; reject an empty value.
Email	If collected, check a basic email format and define duplicate policy.
Year of study	If collected, require a valid integer within the course's allowed range.
Event name	Required and should not contain only spaces.
Event date	Require a consistent date format and validate the selected date.
Capacity	If used, require a non-negative or positive integer according to project rules.
Registration IDs	Student and event must already exist.
Duplicate registration	Optionally prevent repeated student_id + event_id pairs.

14.3 Primary-key integrity

The primary key prevents two rows from sharing the same identifier. Integer primary keys may be generated automatically by SQLite. The application should display the generated identifier only if it is useful to the user.

14.4 Foreign-key integrity

The registration record must reference an existing student and event. In SQLite, foreign-key enforcement should be enabled on each connection using PRAGMA foreign_keys = ON. The schema should define the appropriate FOREIGN KEY clauses.

14.5 SQL injection prevention

SQL injection can occur when untrusted text is directly combined with SQL commands. Parameterised queries use placeholders such as ? and supply values separately. This should be the standard approach for every query that contains user-provided values.

14.6 Privacy and responsible sample data

For classroom demonstrations, use fictional student names and contact details. Do not expose real student records in screenshots or public repositories without permission. If the system is extended for actual college use, it would need access control, secure hosting, backups and a clear data-retention policy.

CHAPTER 15

TESTING PLAN AND EXPECTED RESULTS

Testing checks whether the application behaves as expected for normal inputs and invalid cases. The following table is a proposed test plan. Mark each test as Pass or Fail only after running it on the actual application; expected results are not proof that the test has already been completed.

Test ID	Test case	Expected result	Actual status
T01	Open application	Home page loads without an error.	To be tested
T02	Add a valid event	Event is saved and appears in the event list.	To be tested
T03	View events	Saved event details are displayed.	To be tested
T04	Update event venue	Only the selected event is updated.	To be tested
T05	Delete a test event	Test event is removed or a safe error is shown.	To be tested
T06	Add a valid student	Student is saved and displayed.	To be tested
T07	Submit empty required field	Validation message appears and invalid data is not saved.	To be tested
T08	Register valid student for valid event	Registration is saved and linked correctly.	To be tested
T09	Register with missing ID	Database or application rejects invalid relationship.	To be tested
T10	View registration list	Registration records are displayed.	To be tested
T11	Repeat a registration	Behaviour follows the defined duplicate-registration rule.	To be tested
T12	Use special characters in text	Input is treated as data and query still behaves safely.	To be tested

15.1 Functional testing

Functional testing verifies that the main buttons and forms work correctly. Each operation should be checked using a known test record, and the result should be confirmed in the user interface or database table.

15.2 Integration testing

Integration testing checks the connection between Streamlit, Python and SQLite. A record entered through the interface should be persisted in the database and should remain available when the list is refreshed or the application is reopened, subject to the database path used.

15.3 Test evidence

Recommended evidence includes a screenshot of the initial dashboard, a newly added event, an updated record, a student record, a successful registration and a final table view. Screenshots should be captured from the actual running application and should not be fabricated.

CHAPTER 16

SAMPLE DATA AND DEMONSTRATION SCENARIOS

16.1 Sample event records

event_id	event_name	category	date	venue	capacity
1	Python Workshop	Technical	2026-11-10	Computer Lab	60
2	Cultural Day	Cultural	2026-11-18	Auditorium	250
3	Sports Meet	Sports	2026-12-02	Sports Ground	300

16.2 Sample student records

student_id	full_name	email (fictional)	department	year
1	Aarav Patil	aarav.example@example.com	AI-DS	3
2	Neha Sharma	neha.example@example.com	AI-DS	3
3	Kabir Joshi	kabir.example@example.com	Computer Engineering	2

The names and event details above are illustrative examples for explaining the database. They are not claimed to be records already stored in the user's application.

16.3 Sample registrations

registration_id	student_id	event_id	registration_date	status
1	1	1	2026-10-25	Registered
2	2	1	2026-10-25	Registered
3	3	2	2026-10-25	Registered

16.4 Demonstration scenario A: create an event

Open the event-management page, enter a fictional event name and other required fields, and submit the form. Confirm that a new event row appears in the event table. Record the actual outcome and capture a screenshot.

16.5 Demonstration scenario B: register a student

Choose an existing student and an existing event, submit the registration form and open the registration list. Confirm that the registration refers to the correct student and event. If the application displays names through a join, verify that the displayed names match the selected IDs.

16.6 Demonstration scenario C: update and delete

Edit a test event and verify the change. Then delete only a disposable test record and confirm that the expected row is removed. Avoid deleting a parent record that has dependent registrations unless the application explicitly handles the related records safely.

CHAPTER 17

USER INTERFACE / SCREENSHOT CHECKLIST

17.1 Interface design

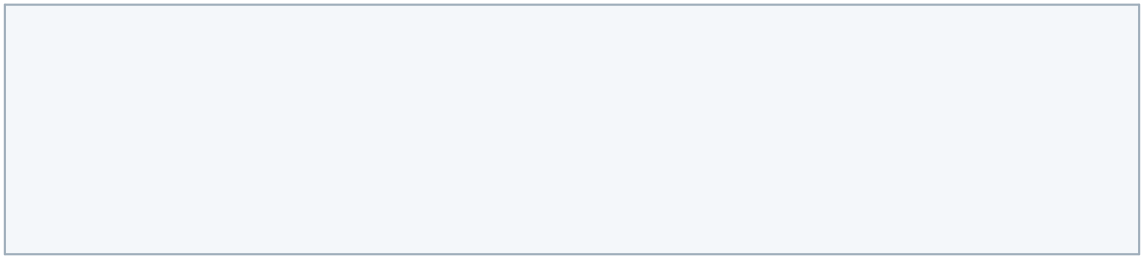
A Streamlit interface can use a sidebar or navigation controls to separate Events, Students and Registrations. Forms should be arranged in a clear order, while tables should display relevant fields. Success and error messages should be easy to notice. Keep the visual design consistent across modules.

17.2 Screenshots to insert

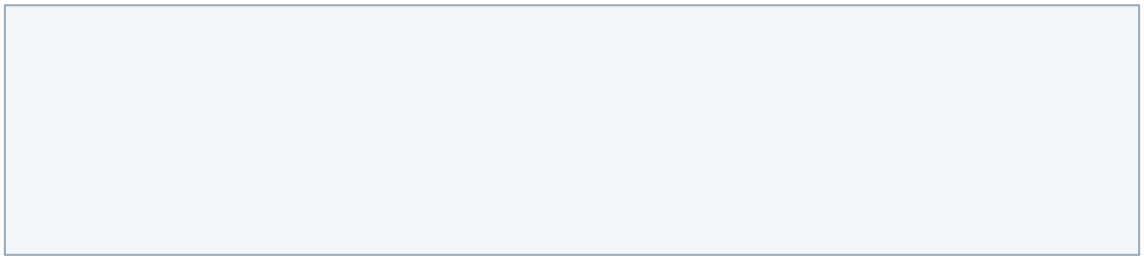
No.	Screenshot required	Evidence / caption
1	Home or dashboard page	Shows application title and available modules.
2	Manage Events page	Shows the event list and add-event form.
3	Event successfully added	Shows a new test event in the list.
4	Event successfully updated	Shows the edited value after saving.
5	Students page	Shows a student record and available controls.
6	Registration form	Shows selection of a student and event.
7	Registration list	Shows the saved registration.
8	Database table or joined output	Shows the records stored in SQLite.
9	Validation message	Shows a safe response to missing or invalid input.
10	Delete demonstration	Shows the result after deleting a disposable test record.

17.3 Screenshot placeholders

Insert Screenshot 1 here: Home / Dashboard



Insert Screenshot 2 here: Events / Students / Registrations



Important: replace the placeholder areas with screenshots from your own running project before submission.

CHAPTER 18

INSTALLATION AND EXECUTION GUIDE

18.1 Preparation

Install Python and choose a code editor such as VS Code or PyCharm. Keep the project files in one folder. The commands below are a general example for a Streamlit and SQLite project; adapt them to the folder and file names used by the actual project.

18.2 Create a project folder

```
CollegeEventRegistration/
    app.py
    requirements.txt
    database.py        (if used)
    schema.sql         (if used)
    README.md          (optional)
```

18.3 Install Streamlit

```
python -m pip install streamlit
```

If a requirements.txt file is available and correct, install the project's listed dependencies with:

```
python -m pip install -r requirements.txt
```

18.4 Start the application

```
streamlit run app.py
```

After the command starts successfully, Streamlit normally displays a local URL in the terminal. Open that URL in a web browser. If the project uses a different entry-point filename, run the command with that filename instead.

18.5 Common problems and fixes

Problem	Possible check
'streamlit' not recognised	Use python -m streamlit run app.py and confirm Streamlit is installed in the active environment.
Module not found	Install the required package in the same Python environment used to run the app.
Database file not found	Check the database path and confirm whether the app creates the file automatically.
Table does not exist	Run the schema creation step if the project includes one.
Changes are not visible	Refresh the page and verify the database path and SQL commit behaviour.
Port already in use	Stop the earlier Streamlit process or use the port option supported by Streamlit.

18.6 Data backup

Before major testing, keep a copy of the SQLite database file if it contains important test records. Do not overwrite or delete a database file without confirming that it is the intended file.

CHAPTER 19

SECURITY, LIMITATIONS AND FUTURE SCOPE

19.1 Basic security practices

- Use parameterised SQL for user-supplied values.
- Validate input in the application and use database constraints where appropriate.
- Keep real student contact details out of public screenshots and repositories.
- Restrict access if the system is deployed beyond a classroom demonstration.
- Keep backup copies of important data and test restore procedures.
- Show friendly error messages rather than exposing internal traces.

19.2 Limitations of the basic academic version

The original project description focuses on local database connectivity and CRUD operations. Unless separately implemented, the system should not be assumed to provide secure login, role-based permissions, automatic notifications, attendance scanning, payment processing, large-scale concurrent access or production-grade backup and recovery.

SQLite is well suited to a small local demonstration. If many users need to write data at the same time or the college requires a centrally managed system, a server-based database may be more appropriate. The current report does not claim that these extensions have already been implemented.

19.3 Future enhancements

Enhancement	Possible benefit
Login and roles	Separate access for students, event coordinators and administrators.
Event capacity checking	Warn or prevent registration when an event reaches its limit.
Search and filters	Find events, students and registrations more quickly.
Email notifications	Send registration confirmation and event reminders.
QR-code attendance	Support faster attendance recording at an event.
CSV / Excel export	Allow staff to prepare reports and summaries.
Attendance module	Track registered, attended and absent participants.
Server database	Support a larger deployment with multiple users.
Audit history	Track important changes for accountability.

19.4 Development roadmap

A practical development sequence is: first ensure that the existing CRUD features work; then add validation and duplicate checks; next implement search and capacity checking; then introduce authentication and reporting; finally consider hosting and integrations after privacy and security requirements are reviewed.

CHAPTER 20

CONCLUSION AND LEARNING OUTCOMES

20.1 Conclusion

The College Event Registration System is a practical example of how a relational database can support the management of college events and student registrations. The design separates students, events and registrations into related tables and uses primary and foreign keys to represent their relationships. Python provides the application logic, Streamlit provides the user interface and SQLite stores the data.

The project focuses on core DBMS learning outcomes: creating a schema, writing SQL statements, connecting an application to a database, performing CRUD operations, joining related tables and maintaining data integrity. It provides a suitable foundation for future improvements such as search, capacity checking, reporting and authentication.

The final evaluation should be based on the actual running application. The team should demonstrate each CRUD operation, verify the saved database records and add genuine screenshots in the relevant sections of this report.

20.2 Learning outcomes

• Understanding tables, rows, columns and relational database design.
• Identifying primary keys and foreign keys.
• Representing a many-to-many relationship using a junction table.
• Applying basic normalisation principles.
• Writing INSERT, SELECT, UPDATE and DELETE statements.
• Using SQL JOIN and aggregate functions.
• Connecting Python to SQLite.
• Designing simple forms and tables with Streamlit.
• Validating inputs and handling database errors.
• Planning tests and documenting results honestly.

20.3 Team demonstration checklist

Before submission, confirm that the application starts correctly, the database is created or located correctly, each module can display records, a test event can be inserted and updated, a test student can be inserted, a registration can be created, and the results can be shown in the interface. Replace all test-status labels and screenshot placeholders with verified evidence.

20.4 Final note

This report is based on the five-page College Event Registration System template supplied for the project and expands its sections into a longer report format. Any implementation-specific details should be checked against the actual code before the report is submitted.

REFERENCES

The following references support the technologies and database concepts discussed in this report. Use the versions and documentation pages relevant to the environment used for the actual project.

1. Silberschatz, A., Korth, H. F., and Sudarshan, S. *Database System Concepts*. McGraw-Hill.

2. SQLite Documentation. https://www.sqlite.org/docs.html

3. Python Documentation — sqlite3 module. https://docs.python.org/3/library/sqlite3.html

4. Streamlit Documentation. https://docs.streamlit.io/

5. SQLite SQL Language Reference — CREATE TABLE, INSERT, SELECT, UPDATE, DELETE and constraints. https://www.sqlite.org/lang.html

6. Course notes and teaching material for Database Management Systems (DBMS), G. H. Raisoni International Skill Tech University.

7. Project source files: app.py, database connection module, SQL schema and other files supplied or created by the student team. Add the exact repository URL here if the project is published.

END OF REPORT

APPENDIX

VIVA QUESTIONS AND ANSWERS

1. What is a DBMS?

A Database Management System is software used to store, organise, retrieve and manage data.

2. What is a relational database?

It stores data in tables consisting of rows and columns, with relationships defined using keys.

3. What is a primary key?

A primary key uniquely identifies each row in a table and should not contain duplicate values.

4. What is a foreign key?

A foreign key references a key in another table and helps maintain valid relationships between records.

5. Why is a registrations table needed?

It links students and events. One student can register for many events and one event can have many students.

6. What does CRUD mean?

CRUD means Create, Read, Update and Delete.

7. What is SQLite?

SQLite is a lightweight relational database engine that stores the database in a file and does not require a separate database server for a basic local application.

8. Why use Streamlit?

Streamlit helps create a browser-based interface using Python, including forms, buttons and data tables.

9. What is normalisation?

Normalisation organises tables to reduce repeated data and avoid insertion, update and deletion anomalies.

10. What is a JOIN?

A JOIN combines related rows from two or more tables using a matching condition, such as student_id or event_id.

11. Why use parameterised queries?

They pass input values separately from SQL text, improving safety and helping prevent SQL injection.

12. What is the purpose of testing?

Testing checks whether the application performs expected operations and handles invalid inputs correctly.

13. What is a database constraint?

A constraint is a rule such as NOT NULL, UNIQUE, CHECK or FOREIGN KEY that restricts invalid data.

14. What is a possible future feature?

Login with roles, event capacity checking, email confirmations, QR attendance or exportable reports can be considered for future development.

15. What should be shown during the demonstration?

Show the application, database connection, event and student records, all CRUD operations and a registration linked to valid student and event records.